

Снижение размерности

Сингулярное разложение и метод главных компонент

Сингулярное разложение

Singular value decomposition (SVD) / сингулярное разложение матрицы X - это способ создать для данных большой размерности оптимальное приближение меньшей размерности. Таким образом, найдётся несколько доминантных паттернов, которые могут эффективно объяснить дисперсию данных в X .

Сингулярное разложение предполагает разложение матрицы X на три компонента:

$$X = U * W * V^T$$

Сингулярное разложение

Возьмем корпус предложений и векторизуем их:

text1 = 'Бегемот шел по улице'

text2 = 'Бегемот - черный кот'

text3 = 'Черный кот шел по улице'

text4 = 'Маргарита несет цветы'

text5 = 'Маргарита - ведьма'

	T1	T2	T3	T4	T5
бегемот	1	1	0	0	0
ведьма	0	0	0	0	1
кот	0	1	1	0	0
маргарита	0	0	0	1	1
несет	0	0	0	1	0
по	1	0	1	0	0
улице	1	0	1	0	0
цветы	0	0	0	1	0
черный	0	1	1	0	0
шел	1	0	1	0	0

Тогда матрицы U , W и V^T примут вид:

-0.33	0.00	-0.15	0.00	0.93	0.00	0.00	0.00	0.00	0.00
0.00	0.28	0.00	0.72	-0.00	0.30	0.30	0.24	-0.26	0.30
-0.38	0.00	-0.55	0.00	-0.22	-0.18	-0.18	0.00	-0.63	-0.18
0.00	0.72	0.00	0.28	-0.00	-0.30	-0.30	-0.24	0.26	-0.30
-0.00	0.45	-0.00	-0.45	0.00	0.29	0.29	-0.53	-0.25	0.29
-0.45	-0.00	0.35	-0.00	-0.10	0.67	-0.33	-0.00	-0.00	-0.33
-0.45	-0.00	0.35	-0.00	-0.10	-0.33	0.67	-0.00	-0.00	-0.33
-0.00	0.45	-0.00	-0.45	0.00	0.01	0.01	0.77	-0.01	0.01
-0.38	0.00	-0.55	0.00	-0.22	0.18	0.18	0.00	0.63	0.18
-0.45	-0.00	0.35	-0.00	-0.10	-0.33	-0.33	-0.00	-0.00	0.67

*

2.90	0.0	0.0	0.0	0.0
0.0	1.90	0.0	0.0	0.0
0.0	0.0	1.55	0.0	0.0
0.0	0.0	0.0	1.18	0.0
0.0	0.0	0.0	0.0	1.09

*

-0.58	-0.37	-0.73	0.00	0.00
-0.00	0.00	-0.00	0.85	0.53
0.58	-0.82	-0.04	0.00	0.00
0.00	0.00	-0.00	-0.53	0.85
0.58	0.44	-0.69	0.00	-0.00

-0.33	0.00	-0.15	0.00	0.93	0.00	0.00	0.00	0.00	0.00
0.00	0.28	0.00	0.72	-0.00	0.30	0.30	0.24	-0.26	0.30
-0.38	0.00	-0.55	0.00	-0.22	-0.18	-0.18	0.00	-0.63	-0.18
0.00	0.72	0.00	0.28	-0.00	-0.30	-0.30	-0.24	0.26	-0.30
-0.00	0.45	-0.00	-0.45	0.00	0.29	0.29	-0.53	-0.25	0.29
-0.45	-0.00	0.35	-0.00	-0.10	0.67	-0.33	-0.00	-0.00	-0.33
-0.45	-0.00	0.35	-0.00	-0.10	-0.33	0.67	-0.00	-0.00	-0.33
-0.00	0.45	-0.00	-0.45	0.00	0.01	0.01	0.77	-0.01	0.01
-0.38	0.00	-0.55	0.00	-0.22	0.18	0.18	0.00	0.63	0.18
-0.45	-0.00	0.35	-0.00	-0.10	-0.33	-0.33	-0.00	-0.00	0.67

*

2.90	0.0	0.0	0.0	0.0
0.0	1.90	0.0	0.0	0.0
0.0	0.0	1.55	0.0	0.0
0.0	0.0	0.0	1.18	0.0
0.0	0.0	0.0	0.0	1.09

*

-0.58	-0.37	-0.73	0.00	0.00
-0.00	0.00	-0.00	0.85	0.53
0.58	-0.82	-0.04	0.00	0.00
0.00	0.00	-0.00	-0.53	0.85
0.58	0.44	-0.69	0.00	-0.00

Давайте посмотрим, сколько информации нам даст перемножение только первых двух компонентов этих матриц.

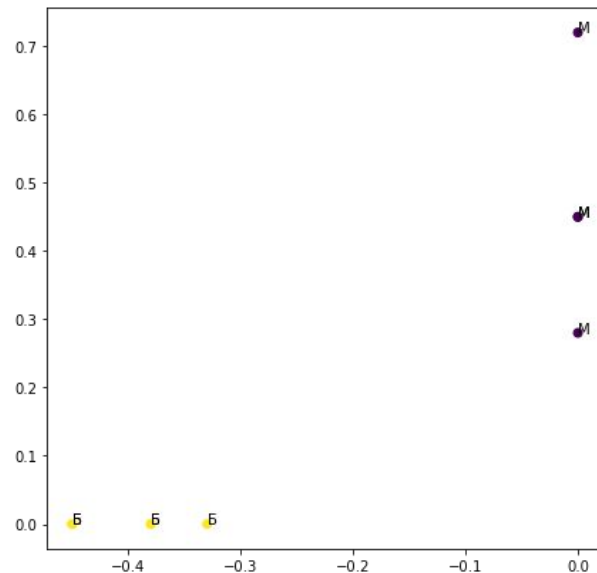
	T1	T2	T3	T4	T5
бегемот	1	1	0	0	0
ведьма	0	0	0	0	1
кот	0	1	1	0	0
маргарита	0	0	0	1	1
несет	0	0	0	1	0
по	1	0	1	0	0
улице	1	0	1	0	0
цветы	0	0	0	1	0
черный	0	1	1	0	0
шел	1	0	1	0	0

	T1	T2	T3	T4	T5
бегемот	0.55	0.36	0.69	0.00	0.00
ведьма	-0.00	-0.00	-0.00	0.45	0.28
кот	0.64	0.41	0.80	0.00	0.00
маргарита	-0.00	0.00	-0.00	1.17	0.72
несет	0.00	0.00	-0.00	0.72	0.45
по	0.75	0.49	0.95	-0.00	-0.00
улице	0.75	0.49	0.95	-0.00	-0.00
цветы	0.00	0.00	-0.00	0.72	0.45
черный	0.64	0.41	0.80	0.00	0.00
шел	0.75	0.49	0.95	-0.00	-0.00

Посмотрите на исходную матрицу и на новую и сделайте выводы о том, как изменилась представленная информация.

Попробуем изобразить два компонента матрицы U на плоскости:

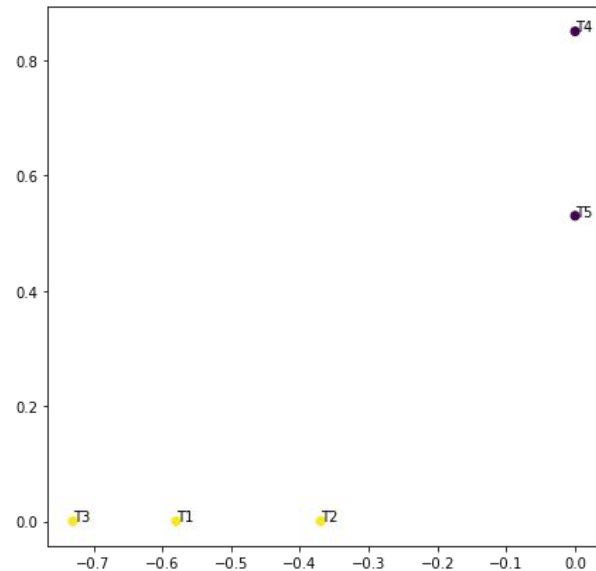
бегемот	-0.33	0.00	-0.15	0.00	0.93	0.00	0.00	0.00	0.00	0.00
ведьма	0.00	0.28	0.00	0.72	-0.00	0.30	0.30	0.24	-0.26	0.30
кот	-0.38	0.00	-0.55	0.00	-0.22	-0.18	-0.18	0.00	-0.63	-0.18
маргарита	0.00	0.72	0.00	0.28	-0.00	-0.30	-0.30	-0.24	0.26	-0.30
несет	-0.00	0.45	-0.00	-0.45	0.00	0.29	0.29	-0.53	-0.25	0.29
по	-0.45	-0.00	0.35	-0.00	-0.10	0.67	-0.33	-0.00	-0.00	-0.33
улице	-0.45	-0.00	0.35	-0.00	-0.10	-0.33	0.67	-0.00	-0.00	-0.33
цветы	-0.00	0.45	-0.00	-0.45	0.00	0.01	0.01	0.77	-0.01	0.01
черный	-0.38	0.00	-0.55	0.00	-0.22	0.18	0.18	0.00	0.63	0.18
шел	-0.45	-0.00	0.35	-0.00	-0.10	-0.33	-0.33	-0.00	-0.00	0.67



Можно видеть, что слова линейно разделимы по темам документов, а отдельные слова (“Бегемот” и “кот”, “по”, “улице” и “шел”) расположены очень близко.

А теперь изобразим два компонента матрицы V^T :

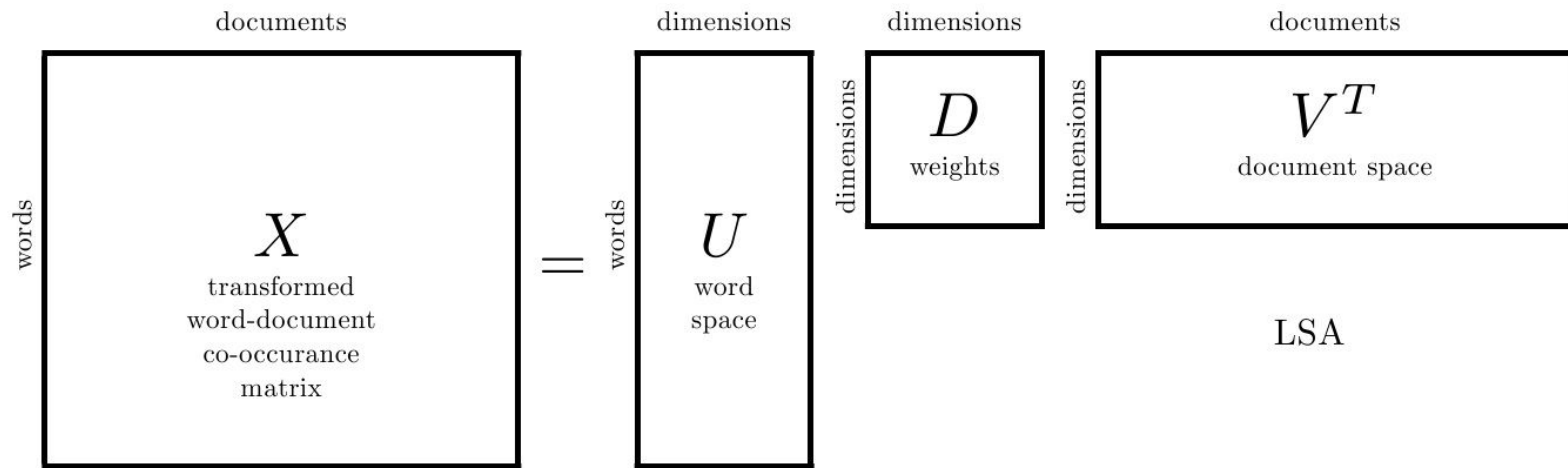
T1	T2	T3	T4	T5
-0.58	-0.37	-0.73	0.00	0.00
-0.00	0.00	-0.00	0.85	0.53
0.58	-0.82	-0.04	0.00	0.00
0.00	0.00	-0.00	-0.53	0.85
0.58	0.44	-0.69	0.00	-0.00



Классы документов также линейно разделимы.

Латентно-семантический анализ (LSA)

Метод латентно-семантического анализа основан на сингулярном разложении матриц текстов. При помощи него можно посмотреть на распределение топиков.



Метод главных компонент (principal component analysis, PCA)

Метод главных компонент - это статистическая интерпретация сингулярного разложения.

Алгоритм получения главных компонент матрицы X :

- 1) Получим матрицу X' - усредненную матрицу X ;
- 2) Вычислим ковариационную матрицу $C(X)$;
- 3) Получим собственные вектора и собственные значения C ;
- 4) Отсортируем собственные вектора по убыванию собственных значений, получив, таким образом, главные компоненты исходной матрицы.

Метод главных компонент (principal component analysis, PCA)

Метод главных компонент - это статистическая интерпретация сингулярного разложения.

Алгоритм получения главных компонент матрицы X при помощи сингулярного разложения:

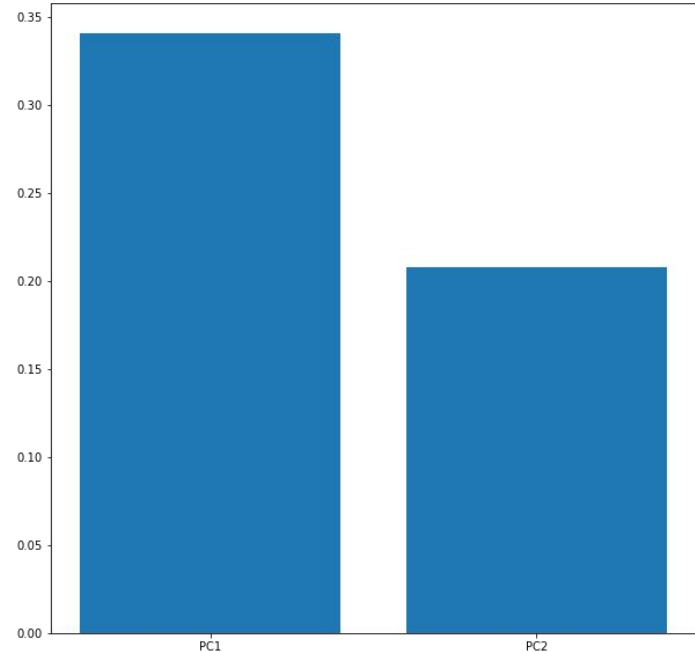
- 1) Получим матрицу X' - усредненную матрицу X ;
- 2) Вычислим $B = X - X'$;
- 3) Получим сингулярное разложение $B = UWV^T$;
- 4) Произведение UW из предыдущего пункта будет являться главными компонентами, при этом ненулевые значения W будут объяснять, сколько дисперсии объясняет каждая компонента.

Метод главных компонент (principal component analysis, PCA)

Скалярное произведение исходной матрицы на N главных компонент дает уменьшенную матрицу размера (число вхождений, N).

Первые несколько главных компонент, как правило, объясняют большую часть дисперсии в данных.

Например, справа две главные компоненты объясняют более 50% дисперсии.



РСА: преимущества и недостатки

Преимущества:

- Большая часть информации сохраняется;
- Ускоряет обучение и предотвращает переобучение.

Недостатки:

- Часть информации неизбежно теряется;
- Находит только линейные зависимости в данных;
- Главные компоненты мало связаны с реальными признаками.

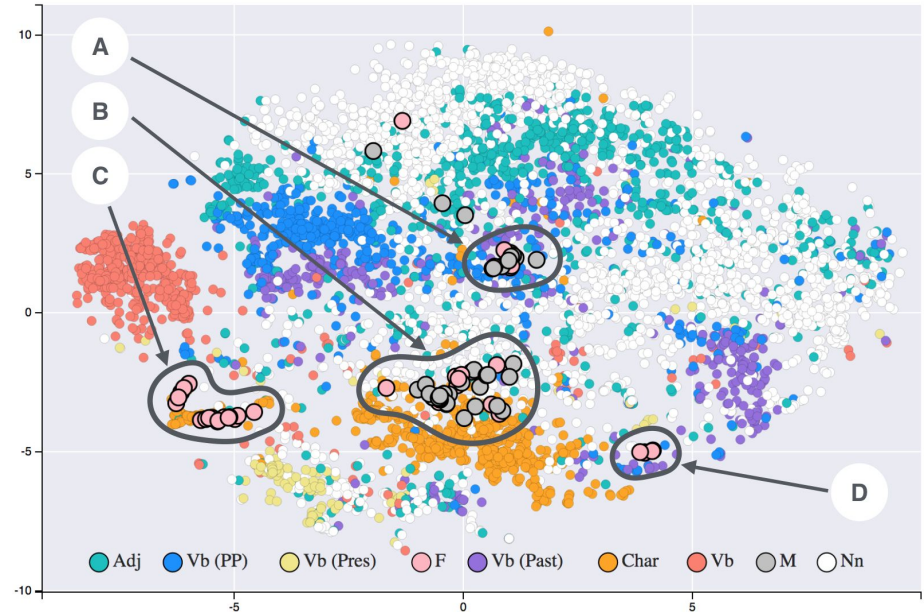
Имплементация метода главных компонент вручную:

<https://colab.research.google.com/drive/1EZOKSzKZ7qmmRIQDRfLiczOmqq0t3oVR?usp=sharing>

t-SNE

t-SNE

t-SNE (t-distributed Stochastic Neighbor Embedding) – это метод снижения размерности и визуализации данных, который позволяет сохранить локальные структуры данных и обнаруживать нелинейные зависимости.



Источник: Siobhán Grayson - T-SNE visualisation of word embeddings generated using 19th century literature, <https://commons.wikimedia.org/w/index.php?curid=64541584>

t-SNE

- t-SNE начинается с расчета условных вероятностей сходства объектов в исходном пространстве. Более сходные (близкие) точки имеют большую вероятность быть соседями;
- Затем, t-SNE строит распределение вероятностей для целевого пространства, таким образом, чтобы объекты, которые близки в исходном пространстве, имели высокие вероятности быть близкими и в сниженном пространстве.

t-SNE

Гиперпараметры:

- Количество компонент;
- Перплексия (perplexity): определяет, сколько соседей учитываются в расчете условных вероятностей сходства. Большая перплексия приводит к усреднению вероятностей и созданию более глобальной структуры, в то время как маленькая перплексия подчеркивает локальную структуру.

t-SNE

Преимущества:

- Обнаруживает нелинейные зависимости в данных;
- Сохраняет локальные структуры данных.

Недостатки:

- Полезен для визуализации многомерных данных, но не очень подходит для извлечения признаков;
- Вычислительная сложность.

t-SNE: ЧТО СТОИТ ПОМНИТЬ

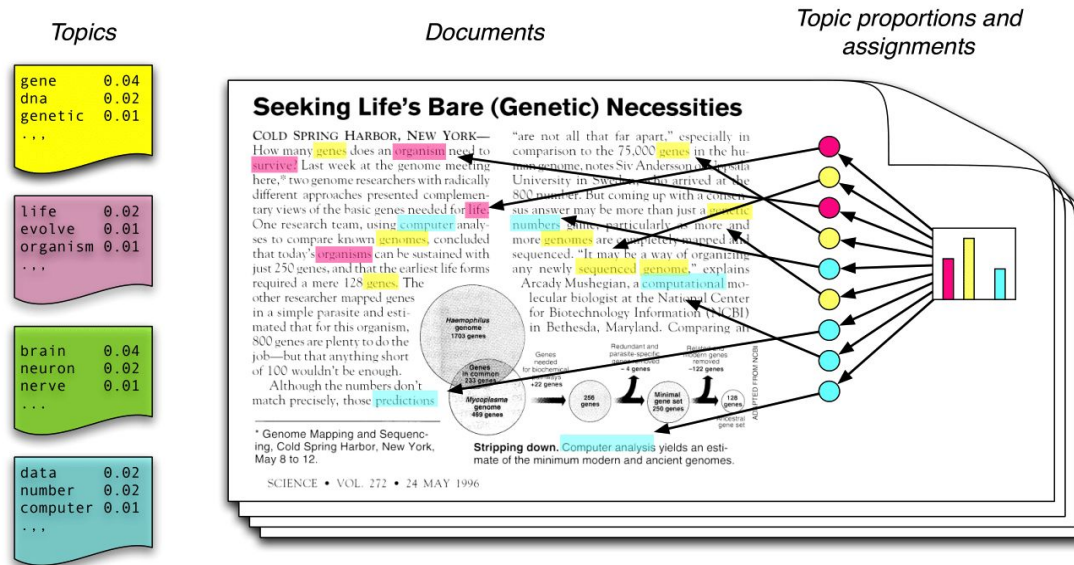
Выжимка из этой статьи: <https://distill.pub/2016/misread-tsne/>.

1. Гиперпараметры сильно влияют на результат;
2. Размеры кластеров t-SNE ничего не значат;
3. Расстояния между кластерами тоже малозначимы;
4. Шум не всегда выглядит, как шум (в зависимости от гиперпараметров вы можете увидеть нечто, похожее на кластеры, в случайных распределениях);
5. Иногда вы можете увидеть формы кластеров, которых на самом деле нет в данных;
6. Вам может понадобиться более одного графика, чтобы оценить топологию.

Латентное размещение Дирихле

Latent Dirichlet Allocation

Позволяет поделить текст на заданное количество тем, основываясь на идее о том, что каждой теме соответствуют определенные слова.



Latent Dirichlet Allocation

1. Выберем количество тем K , которые должна обнаружить модель.
2. Случайным образом назначим каждому слову в каждом документе какую-то из K тем.
3. Перераспределим темы, используя семплирование по Гиббсу.
4. Повторим шаг 3 до достижения сходимости (стабилизации назначения тем).
5. После сходимости модель выводит две матрицы: (документы, темы) и (темы, слова). Распределение документов по темам указывает на вероятность того, что каждый документ принадлежит каждой из K тем, а распределение тем по словам указывает на вероятность того, что каждая тема генерирует каждое слово в корпусе.

Gibbs sampling

- Мы знаем текущее состояние всех назначений тем для всех слов в корпусе;
- Для каждого слова w в каждом документе d пересчитываем вероятность для каждой темы t быть назначенной этому слову. Вычислим:
 - Долю слов в документе d , принадлежащих теме t , исключая текущее слово = $P(t|d)$
 - Сколько раз слово w было назначено теме t во всём корпусе, исключая текущее слово = $P(w|t)$
- Выбираем новую тему для текущего слова: $P(\text{тема } t \mid \text{текущие назначения тем}) \sim P(t|d) * P(w|t)$

LDA

Преимущества:

- Генерирует семантически значимые топики;
- Может работать с большими коллекциями документов;
- Может использоваться для кластеризации

Недостатки:

- Не может самостоятельно определить количество тем;
- Не смотрит на контекст;
- Метод чувствителен к препроцессингу текстов.

LDA - оценка качества топики

- Перплексия и $\log \text{likelihood}$: меры качества предсказаний на данных, которые модель не видела. Чем меньше перплексия, тем лучше; чем больше $\log \text{likelihood}$, тем лучше;
- Coherence/связность: семантическая связность текстов внутри тем (в sklearn “из коробки” не имплементирована).

Практика:

https://colab.research.google.com/drive/1lh57LCXwsq_yfbzw1tyL_qVrQ1goraYX?usp=sharing

ИСТОЧНИКИ

- PCA:
 - <http://math-info.hse.ru/f/2015-16/ling-mag-quant/lecture-pca.html>
 - <https://youtu.be/FgakZw6K1QQ?si=Kk9b49hdWFw2j3ZQ>
 - <https://youtu.be/fkf4IBRSeEc?si=u-f3DoQX-TYD8W4C>
 - <https://habr.com/ru/articles/304214/>
- LDA:
 - <https://dzen.ru/a/ZZ-8RRmYNTKmCis6>
- LSA:
 - <https://habr.com/ru/articles/110078/>
- Всё вместе:
[https://github.com/nstsj/ML_for_NLP/blob/main/6_dimred/dimred_for_texts\(LDA%2BLSA\).ipynb](https://github.com/nstsj/ML_for_NLP/blob/main/6_dimred/dimred_for_texts(LDA%2BLSA).ipynb)
- Математика (собственные вектора и значения):
https://youtu.be/PFDu9oVAE-g?si=xXEC1Or_Qfxk7KDF